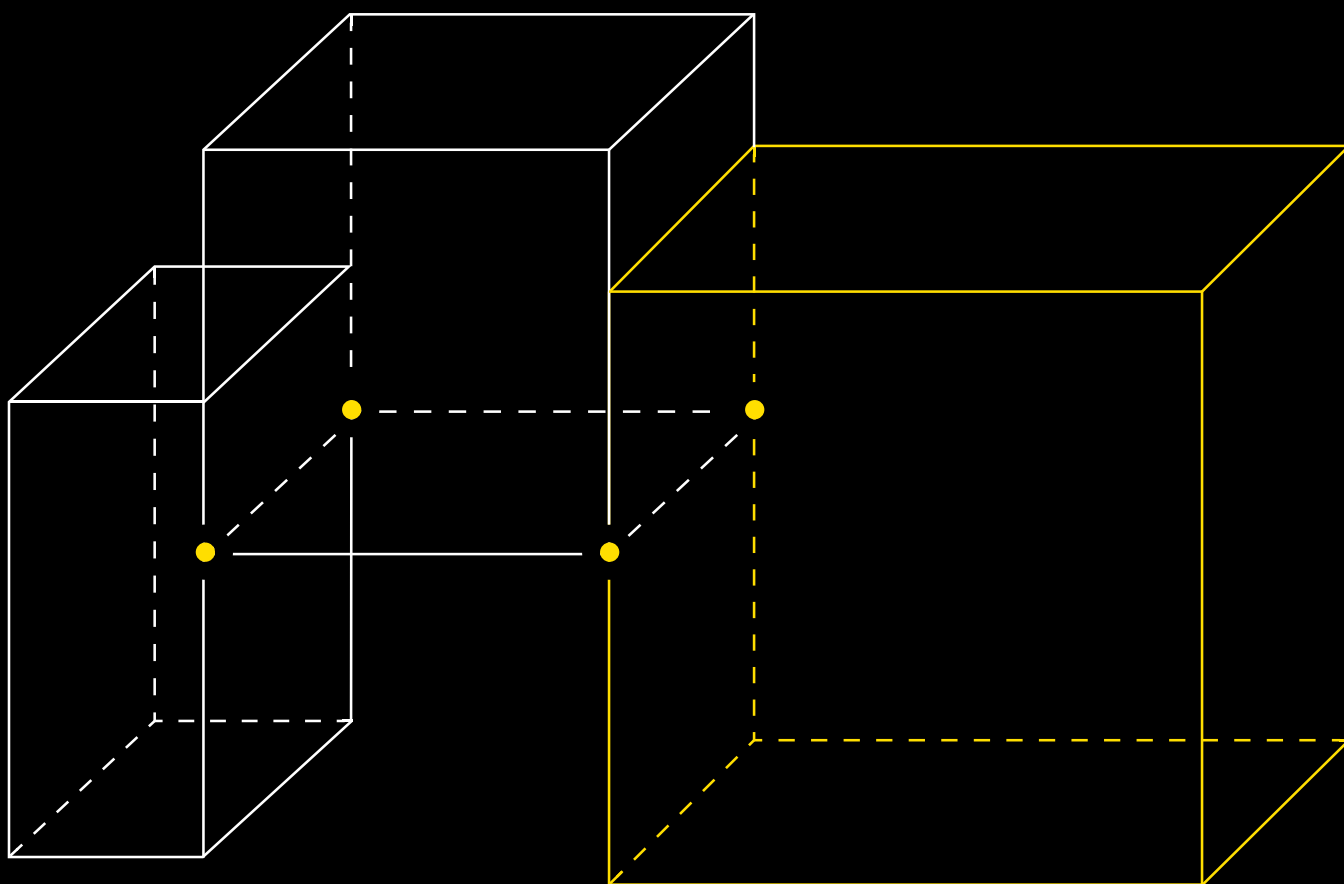


World Models



ИЗБРАННЫЕ ТЕМЫ В ИИ

ДОМАШНЕЕ ЗАДАНИЕ

НЕДЕЛЯ 13

Can a VLM Learn to Imagine?

На лекции мы обсуждали, что все модели мира учат одно уравнение:

$$z_{t+1} = f_{\theta}(z_t, a_t).$$

[VAGEN](#) обучает VLM как реактивную **политику** — модель выдаёт действие $\pi(a|o)$. А что, если обучить VLM как **модель динамики** — предсказывать следующее состояние $f(s'|s, a)$? И использовать предсказания для планирования?

СЕТАП

Среда: FrozenLake



Figure 1: FrozenLake 8×8

Сетка 8×8 , gymnasium. Замёрзшие (безопасные), дыры (провал), цель (успех), старт. Карта, случайная для каждого seed. Четыре действия: Left, Down, Right, Up. Детерминированная (без скольжения).

Модель

Советуем модели семейства Qwen2.5-VL — промпты и пайплайн в этом ДЗ заточены под них. Размер выбирайте под ваш compute.

ЗАДАНИЕ

Три части. Для каждой части оформите отчёт в стиле «гипотеза — дизайн эксперимента — результаты — выводы». Процесс исследования важнее конечного результата.

Часть 1. Реактивный агент (baseline)

Вопрос: насколько далеко можно зайти с реактивной политикой без обучения?

- Запустите zero-shot VLM как реактивного агента на ≥ 20 эпизодах. Спроектируйте system prompt и формат ответа (например, `<answer>Down</answer>`), напишите парсер. Метрики: success rate, format compliance.
- Реализуйте **жадный планировщик**: на каждом шаге он знает точные позиции (player, goal, holes) из ground truth и выбирает действие, максимально приближающее к цели по Manhattan-расстоянию, не наступая на дыру. Это **тот же алгоритм**, что lookahead в части 3, но с идеальными предсказаниями — потолок для жадного планирования (на 8×8 не всегда достигает цели из-за тупиков).

Часть 2. Обучаем модель мира (SFT)

Вопрос: можно ли научить VLM предсказывать динамику среды и что является bottleneck'ом — восприятие или понимание правил?

Данные

Соберите датасет переходов из FrozenLake:

- сгенерируйте ≥ 100 карт (различные seeds);
- на каждой карте прогоните **жадную** и **random**-политику, соберите переходы;
- каждый переход — один пример: `(image_t, action_t) → (next_position, outcome)`;
- `outcome` $\in$ {safe, hole, goal, wall};
- разделите на train/val **по картам (seeds)**: все переходы с одной карты — в одном split. 80% карт — на train, 20% — на val;
- итого ~2000–5000 переходов.

Формат

Зафиксируйте формат ответа модели, например:

```
<prediction>Position: (2, 1). Outcome: safe</prediction>
```

Формат для обучения и eval должен быть **идентичным**.

Эксперимент: восприятие vs динамика

Обучите **две отдельные модели** (два отдельных SFT-запуска с разными данными) и оцените каждую в своём условии. Шаблон промпта **одинаковый**, меняется только вход:

- **Image + GT state text**: модель получает изображение и текстовое описание текущего состояния (позиция игрока, позиции дыр и цели). Нужно предсказать только динамику;
- **Image-only**: модель получает только изображение. Нужно и распознать состояние, и предсказать динамику.

- Метрика: **1-step prediction accuracy** на val seeds. Разделите ошибки через confusion matrix (outcome): safe ↔ hole, неверная позиция, неправильный формат.
- **Калибровочная проверка.** Перед интерпретацией результатов проверьте accuracy на **train seeds**. Если train accuracy << 90%, модель не сошлась, чините обучение до интерпретации.
- **Интерпретация.** Помогает ли GT state text? Что это говорит о bottleneck'e? Сформулируйте гипотезу **до** запуска экспериментов и проверьте её результатами.

Часть 3. Планирование в воображении

Вопрос: помогает ли внешний планировщик, использующий обученную VLM как «движок воображения», и что является bottleneck'ом — качество модели мира или способность планировать?

Qian et al. (2026) показали, что VLM-агенты не умеют надёжно использовать world model как инструмент: редко вызывают симуляцию, неправильно интерпретируют результаты, и производительность в среднем не растёт. Мы обходим проблему управления: **VLM только предсказывает исходы, а планировщик выбирает действие.** Остаётся вопрос: достаточно ли точны предсказания VLM для планирования?

1-step lookahead-планировщик

Для каждого состояния:

1. Для каждого из 4 действий запросите у VLM (или SFT-модели) предсказание: куда переместится игрок и каков исход.
2. Отфильтруйте действия, предсказывающие **hole** или **wall**.
3. Из оставшихся выберите действие с предсказанной позицией, ближайшей к **цели** по Manhattan distance (позицию цели планировщик знает из GT — только предсказание динамики приходит от VLM).
4. Выполните выбранное действие в **реальной** среде.
5. Повторите до конца эпизода.

VLM не планирует сама — планировщик написан в коде, VLM только «воображает» исходы по запросу.

Главное сравнение

Запустите **≥ 30 эпизодов** (одни и те же seeds во всех условиях; reactive- и жадный планировщик из части 1 дорастите до 30). Сравните следующие условия:

1. **Reactive (baseline)** — VLM получает изображение и сразу выдаёт действие (политика из части 1).
2. **Lookahead + zero-shot VLM** — планировщик спрашивает у необученной VLM предсказание для каждого из 4 действий, выбирает лучшее.
3. **Lookahead + SFT (image + text)** — то же, но предсказания от SFT-модели, обученной на изображениях + GT text.
4. **Lookahead + SFT (image-only)** — то же, но предсказания от SFT-модели, обученной только на изображениях.
5. **Lookahead + GT** — то же, но предсказания идеальные из ground truth (потолок).

Метрики: success rate, среднее число шагов, доля попаданий в дыру. Рекомендуем добавить 95% доверительных интервалов (bootstrap) — при $n = 30$ разница в 10–15% может быть статистически незначимой. Оформите результаты в виде таблицы или графика — как удобнее для интерпретации.

Интерпретация

Перед запуском подумайте: какие условия должны быть лучше и почему? После сопоставьте ожидания с результатами. Что оказалось bottleneck'ом: качество восприятия, знание динамики или сам алгоритм планирования?

Бонусы

- **(Бонус А)** Реализуйте **multi-step lookahead** (глубина $H = 2 - 3$). Для $H = 2$ это 16 вариантов, для $H = 3 - 64$. Постройте график **success rate vs глубина планирования**. На какой глубине предсказания начинают расходиться с реальностью?
- **(Бонус В)** Сравните с **CoT-планированием**: в одном промпте попросите VLM рассмотреть все действия, представить исходы и выбрать лучшее. Работает ли «планирование в голове» VLM или только внешний планировщик?
- **(Бонус С)** Повторите эксперимент на **4 × 4 FrozenLake** и сравните с 8 × 8. Как масштаб среды влияет на bottleneck?

КРИТЕРИИ ОЦЕНИВАНИЯ

Что сдать: ноутбук (.ipynb) или репозиторий с кодом, результатами и выводами.

	Баллы	Что оценивается
Часть 1	2	Работающий реактивный агент + жадный планировщик. Таблица с baseline success rate
Часть 2	5	Сбор данных, две обученные модели (image + text, image-only). Prediction accuracy + confusion matrix. Описание обучения (число примеров, гиперпараметры, learning curve). Гипотеза до экспериментов, ответ: восприятие или динамика?
Часть 3	3	Сравнение ≥ 5 условий $\times \geq 30$ эпизодов. Интерпретация: что является bottleneck'ом?
Бонусы	+1 каждый	За каждый выполненный бонус (А, В или С), до +3

Процесс важнее результата. Если SFT-модель мира не работает, это валидный результат, требующий объяснения. Отрицательный результат с интерпретацией ценнее необъяснённого положительного.

СТАТЬИ

- [VAGEN: Training VLM Agents with Multi-Turn RL](#) — фреймворк для обучения VLM-агентов, среда FrozenLake;
- [Current Agents Fail to Leverage World Model as Tool for Foresight](#) — мотивация для части 3: почему VLM не умеют использовать собственное воображение.

ИНСТРУМЕНТЫ И СОВЕТЫ

- **Среда.** `gymnasium`, FrozenLake, `is_slippery=False`, `render_mode="rgb_array"`.
Карты 8 × 8: случайная генерация (`p_hole=0,2`, проверка достижимости BFS). Изображение — `env.render()`, позиции — из `desc` и `env.s`.
- **Compute.** Все числа в задании — ориентиры. Варьируйте под ваш бюджет, но обосновывайте выбор.
- **Токены.** Изображения 8 × 8 генерируют ~600 vision-токенов. `max_length ≥ 1024`, иначе обучение не сойдётся.
- **Seeds.** Фиксируйте `seed` для генерации карт и семплинга модели.